Design and Analysis of Algorithms (22CS2402)

Dr. Bipin Kumar Rai
Professor

Department of Computer Science and Engineering

Dayananda Sagar University, Harohalli, Karnataka.

Email-id: bipin.kumar.rai-cse@dsu.edu.in

Faculty Cabin: A536

AGENDA

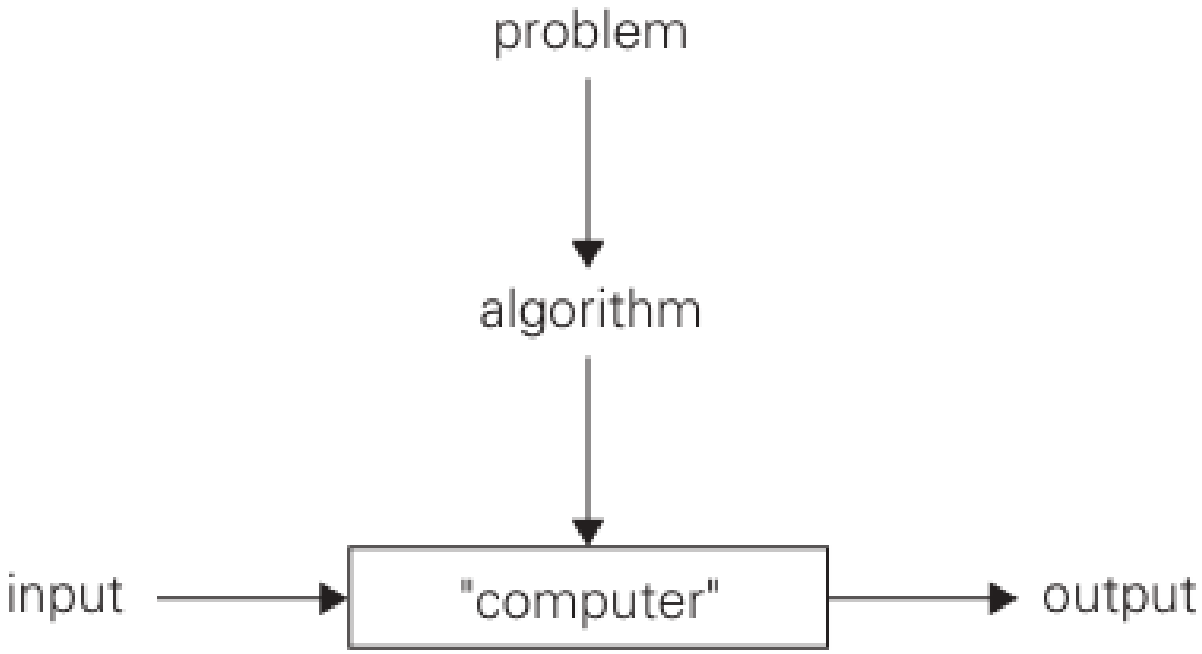
SCHOOL OF ENGINEERING

- What is an Algorithm
- Steps in Designing and Implementing an algorithm
- Important Problem Types

ALGORITHM

- A sequence of Unambiguous instructions for solving a problem, i.e for obtaining the required output for any legitimate input in a finite amount of time.

NOTATION OF THE ALGORITHM



Special Features of Algorithms

- Non-ambiguity.
- Range of Inputs
- The same algorithm can be represented in different ways.
- Several algorithms for solving the same problem.
- Different algorithms with different speeds.

Greatest Common Divisor

- Integers m and n, denoted gcd(m, n), is defined as the largest integer that divides both m and n evenly, i.e., with a remainder of zero.
- Method 1:Descriptive Algorithm

Middle-school Procedure for computing gcd(m, n)

Step 1: Find the prime factors of m.

Step 2: Find the prime factors of n.

Step 3: Identify all the common factors in the two prime expansions found in step 1 and

step2.(If p is a common factors occurring Pm and Pn times in m and n, respectively, it should be repeated min{Pm and Pn} times).

Step 4: Compute the product of all the common factors and return it as the greatest common divisor of the numbers given.

GCD using Euclid's Method

- GCD using Euclid's Method

  gcd(m, n)=gcd(n, m mod n)

Find:

1. GCD(124,12)
2. GCD(160,15)
3. GCD(338,22)
4. GCD(150,8)

Find the GCD(m , n)

- Method 2: Descriptive Algorithm

Euclid's algorithm for computing gcd(m, n)

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

Find the GCD(m , n)

ALGORITHM *Euclid*(m, n)

//Computes gcd(m, n) by Euclid's algorithm

//Input: Two nonnegative, not-both-zero integers m and n

//Output: Greatest common divisor of m and n

while $n \neq 0$ **do**

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

Find the set of all Prime numbers

- Method 1: Middle school level

Find Set of all Prime Numbers within 25.

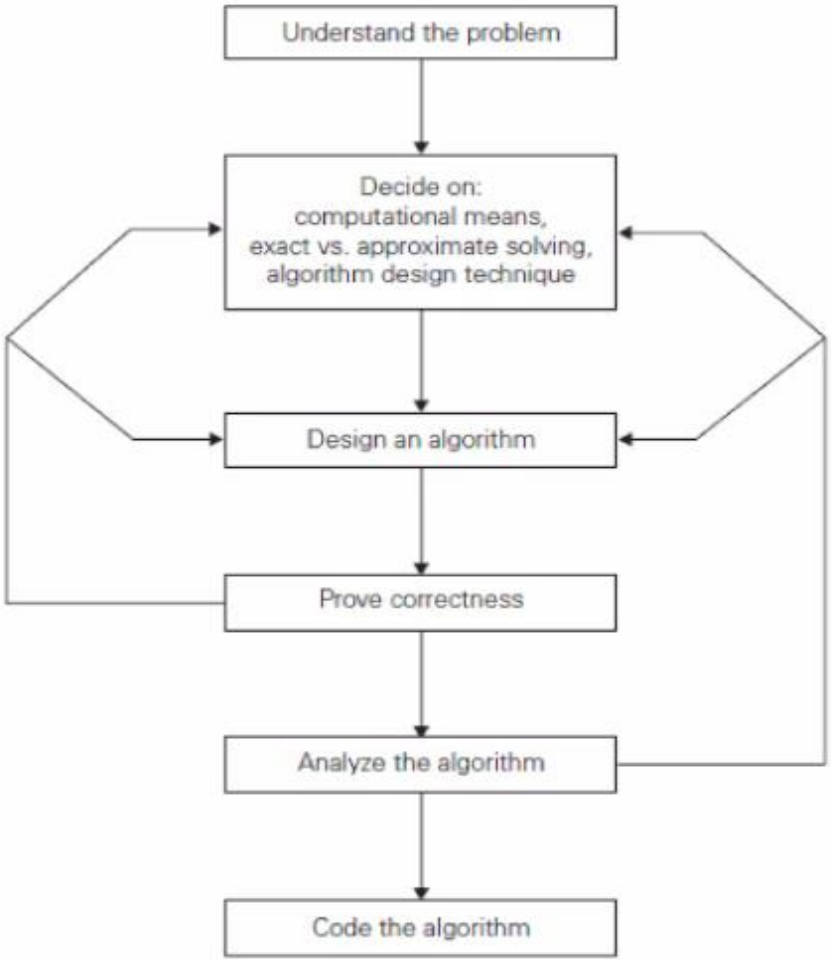
Find the set of all prime numbers

- Method 2: Sieve of Eratosthenes

Find Set of ALL Prime Numbers within 25

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
2	3		5		7		9		11		13		15		17		19		21		23		25
2	**3**		5		7				11		13				17		19				23		25
2	3		**5**		7				11		13				17		19				23		

Fundamentals of Algorithmic Problem Solving/ Algorithm Design & Analysis Process



What does it mean to understand the problem?

- What are the problem objects?
- What are the operations applied to the objects?

Deciding on computational means

- How the objects would be represented?
- How the operations would be implemented?

Design an algorithm

- Build a computational model of the solving process.

Prove correctness

- Correct output for every legitimate input in finite time.
- Based on correct math formula.
- By Induction.

Analyze the algorithm

Efficiency: time and space

Simplicity.

Generality: range of inputs, special cases.

Optimality: no other algorithm can do better

Coding

How the objects and operations in the algorithm are represented in the chosen programming language?

ALGORITHM ANALYSIS

SCHOOL OF ENGINEERING

- Algorithm analysis is a study that provides theoretical estimation of the resources required by an algorithm to solve a specific computational problem.
- Efficiency of the algorithm is measured in terms of time and space complexity.
- Algorithm analysis does not give exact values for time and space complexity , Algorithm analysis does not give exact values for time and space complexity ,however it gives approximate estimate which can be used to study the behavior of algorithms.
- however it gives approximate estimate which can be used to study the behavior of algorithms.

NEED FOR ANALYSIS

SCHOOL OF ENGINEERING

- Software or any application especially those involving large dataset cannot be developed on trail and error basis.
- Programmers cannot waste time and resources in trying out all possible available alternatives to solve the problem at hand.
- Generally there are multiple approaches/methods/algorithms to solve a problem . Algorithm analysis filters out the best/optimal approach of all the available approaches.
- Hence care must be taken at design phase in analyzing and filtering out the best suitable algorithm that solves the problem at hand.

CRITERIA TO ANALYZE AN ALGORITHM

1.**TIME.**

2.SPACE.

3.NETWORK COMMUNICATION: The applications that are web based/internet based/cloud based then the amount of data transferred or network consumption also plays an important role.

4.POWER CONSUMPTION.

5.CPU REGISTERS CONSUMED: if the algorithms are written for device drivers or system software.

TIME COMPLEXITY

SCHOOL OF ENGINEERING

What is Time Complexity?

Time Complexity is the study of the efficiency of algorithms. It tells us how much time is taken by an algorithm to process a given input. Let's understand this concept with the help of an example:

Consider two developers Shubham and Rohan, who created an algorithm to sort 'n' numbers independently. When I made the program run for some input size n, the following results were recorded:

No. of elements (n)	Time Taken By Shubham's Algo	Time Taken By Rohan's Algo
10 elements	90 ms	122 ms
70 elements	110 ms	124 ms
110 elements	180 ms	131 ms
1000 elements	2s	800 ms

We can see that at first, Shubham's algorithm worked well with smaller inputs; however, as we increase the number of elements, Rohan's algorithm performs much better.

Quick Quiz: Who's algorithm is better?

TIME COMPLEXITY

SCHOOL OF ENGINEERING

Note: Time complexity!=Time taken

Time complexity is a mathematical function that define the relationship about how time grows as the input grows.

THINGS TO BE CONSIDERED WHILE DETERMINING COMPLEXITY OF AN ALGORITHM.

1.Always look for the worst case complexity.

2.Always look at complexity for large data.

3.The actual time taken on the machine is not considered.

4.Ignore constants while calculating time complexity.

5.Always ignore less dominating factors.

EFFICIENCY OF THE ALGORITHM DEPENDS ON THE INPUT SIZE

SCHOOL OF ENGINEERING

- One of the instance characteristic for run time complexity is the input size . Larger the input size , larger will be the running time.

Time complexity is measured as a function of

- Input size(n)
- Running time(t(n))
- Different input size n results in different execution time.
- Typically t(n) is the worst case estimate.

METHODS TO COMPUTE THE STEP COUNT

Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size.

Basic operation is the core operation like:

assignment , comparing 2 values, swapping 2 numbers, performing arithmetic operation etc. Basic operation generally resides in a loop.

METHODS TO COMPUTE THE STEP COUNT

- **Method 1: Introduce variable count into programs**
  - Introduce a global variable "count", which is initialized to zero. So each time a statement in the signal program is executed, count is incremented by the step count of that statement.

- **Method 2: Tabular method/Frequency count method/Step count method.**
  - Determine the total number of steps contributed by each statement step per execution × frequency
  - add up the contribution of all statements.

METHOD 1: INTRODUCE VARIABLE COUNT INTO PROGRAMS

SCHOOL OF ENGINEERING

Example 1: Find the time complexity for sum of n elements using count variable method.

Algorithm Sum(x, n)
{
total := 0;
for i:= 1 to n do
total:-total + i
 	
}
	Algorithm Sum(x, n)
	{
	total := 0;
	count:= count+1; /* for assignment */
	for i:= 1 to n do
	{
	count:= count+1; /*for the for loop */
	total:-total + i
	count:= count+1 /* for assignment */
	}
	count:= count+1/* last execution of for */
	}
		Algorithm Sum(x, n)
		{
		count:= count+2;
		/*(2 times)*/
		for i:= 1 to n do
		{
		count:= count+2;
		/*(2n times)*/
		
		}
		}
		
		Total=2n+2 steps
		Time complexity = O(n)

COUNT METHOD:PROBLEM 2

SCHOOL OF ENGINEERING

Algorithm Madd (a,b)
{
for i:= 1 to n do
{
for j:= 1 to n do
{
c[i,j] = a[i,j] + b[i,j];
}
}
}
	Algorithm Madd (a,b)
	{
	for i:= 1 to n do
	{
	count:= count+1;
	for j:= 1 to n do
	{
	count:= count+1;
	c[i,j] = a[i,j] + b[i,j];
	count:= count+1;
	}
	count:= count+1
	}
	count:= count+1;
	}
		Algorithm Madd (a,b)
		{
		count:= count+1; →1
		for i:= 1 to n do
		{
		count:= count+2; →2n
		for j:= 1 to n do
		count:= count+2; →$2n^2$
		}
		}
		
		Total=$2n^2$+ 2n + 1 steps
		Time complexity = O(n^2)

Method 2: Tabular method

Example 1: Find the time complexity for sum of n elements using Tabular method

Statement	S/e	Frequency	Total steps
1. Algorithm Sum(a, n)	0	-	0
2. {	0	-	0
3. s:=0;	1	1	1
4. for i:=1 to n do	1	n+1	n+1
5. s:=s+a[i];	1	n	n
6. return s;	1	1	1
7. }	0	-	0
Total			2n+3 steps

Time complexity = O(n)

Example 2: Matrix addition

Statement	S/e	Frequency	Total Steps
1. Algorithm Madd (a,b)	0	-	-
2.{	0	-	-
3. for i:= 1 to n do	1	n+1	n+1
4. {	0	-	-
5. for j:= 1 to n do	1	n(n+1)	n^2+n
6. {	0	-	-
7. c[i,j] = a[i,j] + b[i,j];	1	n*n	n^2
8. }	0	-	-
9. }	0	-	-
10. }	0	-	-
Total			Total=$2n^2 + 2n + 1$
Time complexity = $O(n^2)$			

- Link for more **problems on time** complexity

https://drive.google.com/file/d/1iztF4UFGIzp3Yo4O67eJlzORcgVip_3G/view?usp=sharing

Space Efficiency

Space Efficiency

- The amount of memory required to run the program completely and efficiently.
- Components that affect space are:

1. Program Space Storing machine program generated by compiler or assembler.
2. Data Space Store constants, variables etc.
3. Stack Space Store the return address along with parameters that are passed to function, local variables etc.

Space Efficiency

To compute space complexity we use two factors:

1. Constant
2. Instance Characteristics
3. The Space requirement is given as: S(P)= C + Sp

S(P) is the space required by a problem P

c is a constant time

S_p is an instance characteristic.

Space Complexity
$S(P)=C+S_P$

FIXED SPACE REQUIREMENTS (C)

Independent of the characteristics of the inputs and outputs

- instruction space
- space for simple variables, fixed-size structured variable, constants

- **VARIABLE SPACE REQUIREMENTS (SP(I))**

depend on the instance characteristic I

- number, size, values of inputs and outputs associated with I.
- recursive stack space, formal parameters, local variables, return address.

REASONS FOR STUDYING SPACE COMPLEXITY

1.If the program is to run on multi-user system , it may be required to specify the amount of memory to be allowed to the program.

2.To know in advance that whether sufficient memory is available to run the program.

3.There may be several possible solutions with different space requirements.

NOTE: Instance characteristics (S_p) varies from problem to problem , so during analyzing the space complexity the sole concentration is on analyzing S_p.

PROBLEMS ON SPACE COMPLEXITY

Example: 1

Algorithm NEC (float x, float y, float z)

{

Return (X + Y +Y * Z + (X + Y +Z)) /(X+ Y) + 4.0;

}

There are no instance characteristics and the space needed by X, Y, Z is independent of instance characteristics,

Therefore S(XYZ) =3+0=3

One space each for X, Y and Z

Space complexity is O(1).

PROBLEMS ON SPACE COMPLEXITY

SCHOOL OF ENGINEERING

Examples: 2

```
Algorithm ADD ( float [], int n)
{
sum = 0.0;
for i=1 to n do
sum=sum+X[i];
return sum;
}
```

Here, at least n words since X must be large enough to hold the n elements to be summed. Here the problem instances are characterized by n, the number of elements to be summed. So,

S(ADD) =3+n

- 3-one each for n, I and sum .
- Where n- is for array X[].
- Space complexity is O(n).

PROBLEMS ON SPACE COMPLEXITY

Example 3

```
Algorithm factorial (int n)
{
if (n == 0)
return 1;
else.
return(n * factorial(n-1));
}
```

Space complexity is O(n).

PROBLEMS ON SPACE COMPLEXITY

Example 4

```
Algorithm Rsum (x,n)
{
if (n<=0) then
return 0;
else
return Rsum((x,n-1) + x[n]);
}
```

Space complexity is O(n).

Order of Growth

- The relative performance of an algorithms depends on input data size N.
- The order of growth is normally determined for larger values of n for the following reasons:

1. The behavior of an algorithm changes as the value of n increases.
2. In real time applications large values of n is encountered.

- If there are multiple input parameters, we will try to reduce them to a single parameter, expressing some parameters in terms of the selected parameter.
- Between two algorithms it is considered that the one having a smaller order of growth is more efficient.
- However, this is true only for large enough input sizes

Dominant Term

- In the running time expression, when n becomes large a term will become significantly larger than the other ones : this is the so-called dominant term

- The growth rate is not affected by
  1. constant factors or
  2. lower-order terms

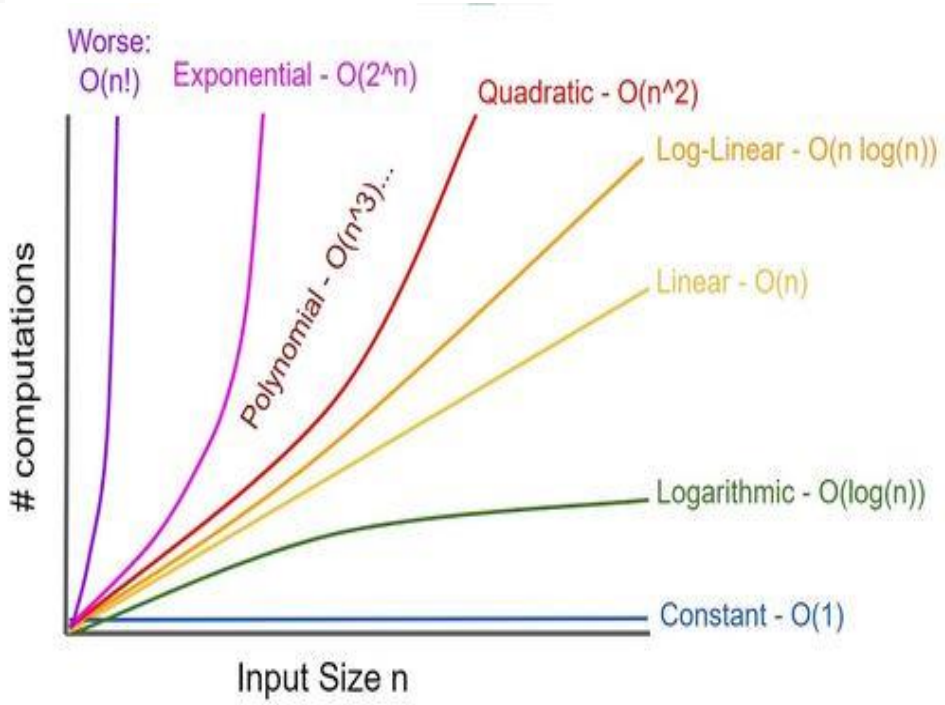
Examples	
$T1(n)=an+b$	Dominant term: an
$T2(n)=a\log n+b$	Dominant term: a log n
$T3(n)=an^2+bn+c$	Dominant term: an^2
$T4(n)=an+b\,n+c\ (a>1)$	Dominant term: an

Rate of growth of function

As N increases order of growth increases

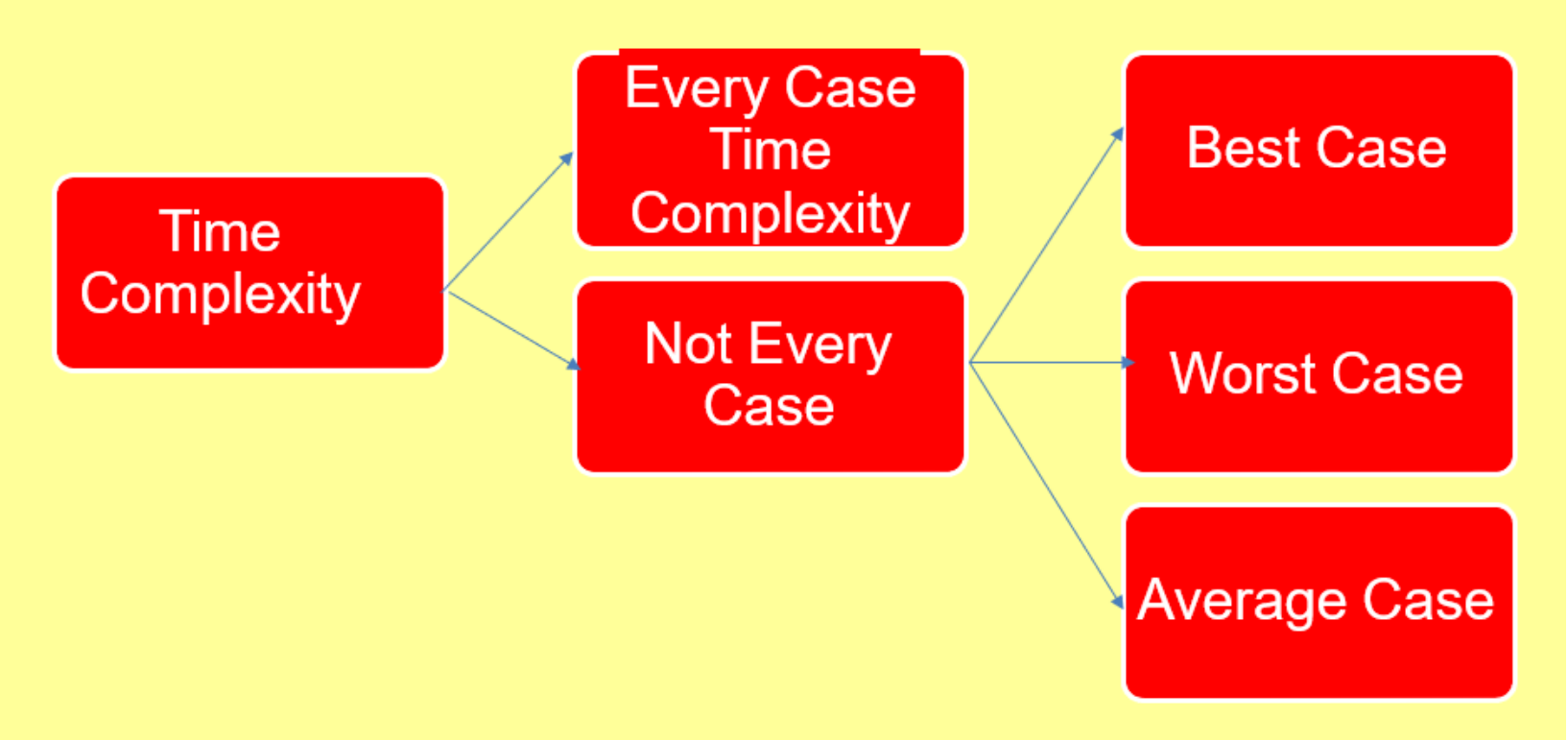
N	log N	N	N log N	N^2	N^3	2^N	N!
1	0	1	0	1	1	2	1
2	1	2	2	4	8	4	2
4	2	4	8	16	64	16	24
8	3	8	24	64	512	256	40320
16	4	16	64	256	4096	65536	high
32	5	32	160	1024	32768	4294967296	very high

Fig 2.2 Values of some of the functions



Every Case Time Complexity

- For a given algorithm, T(n) is every case time complexity if algorithm have to repeat its basic operation every time for given input size n.
- Determination of T(n) is called every case time complexity analysis.



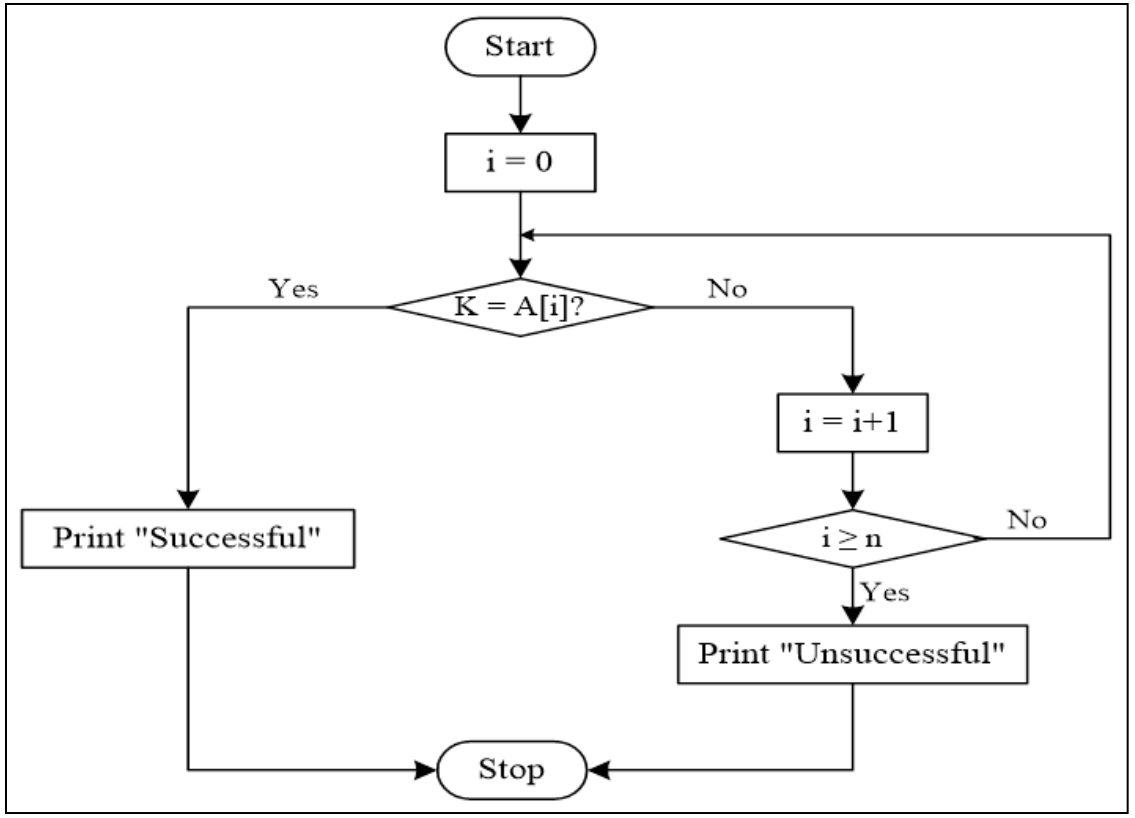
Sequential Search Algorithm

Algorithm
sequential_search(A,n,key)

```
i=0

while i<n && a[i]!=key

    i=i+1

    if i<n

        return i

    else

        return -1
```

LINEAR SEARCH

Linear search is a sequential searching algorithm where we start from one end and check every element of the list until the desired element is found. It is the simplest searching algorithm.



LINEAR SEARCH

3	4	5	6	7	9	10	12	15

- **BEST CASE:** If we want to search an element 3, whether it is present in the array or not. First, A(1) is compared with 3, match occurs. So the number of comparisons is only one. Time complexity is O(1)
- **AVERAGE CASE:** If we want to search an element 7, whether it is present in the array or not.
  - First, A(1) is compared with 7 i.e., (3=7), no match occurs.
  - Next, compare A(2) and 7, no match occurs. Compare A(3) and A(4) with 7, no match occurs. Up to now 4 comparisons takes place.
  - Now compare A(5) and 7 (i.e., 7=7), so match occurs. The number of comparisons is 5
  - Note: If there are n elements, then we require n/2 comparisons.

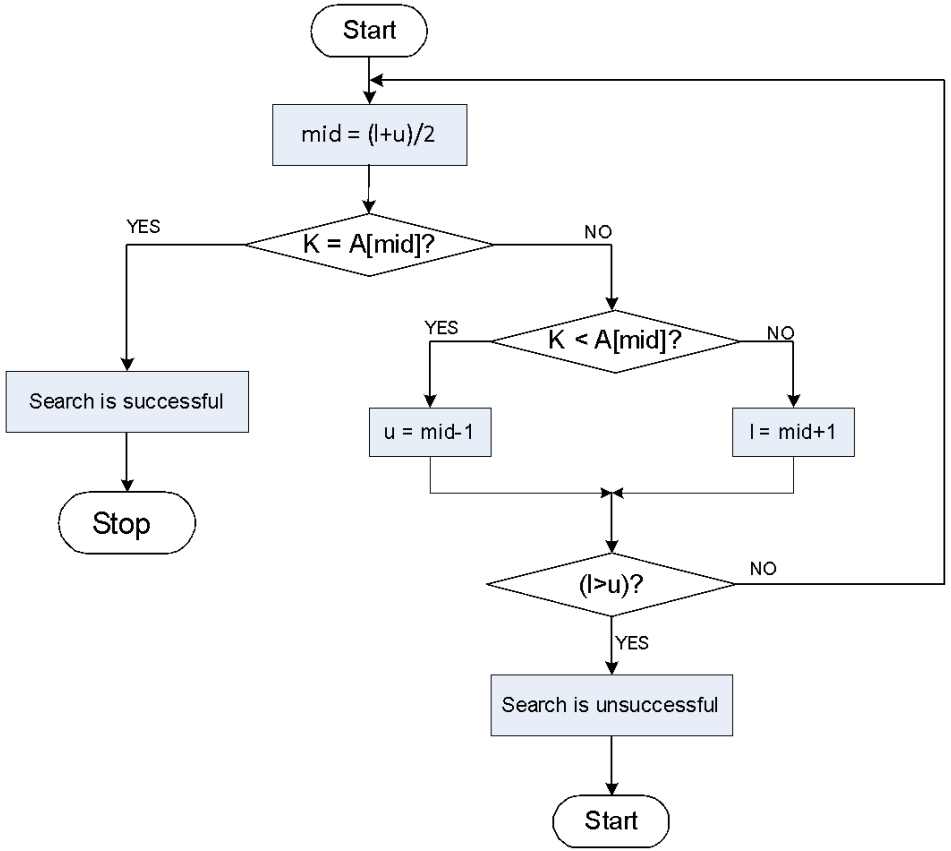
. . Time complexity is O (n/2) = O(n) (we neglect constant)

LINEAR SEARCH

- **WORST CASE:** If we want to search an element 15, whether it is present in the array or not.
  - First, A(1) is compared with 15 (i.e., 3=15), no match occurs.
  - Continue this process until either element is found or the list is exhausted.
  - The element is found at 9th comparison.
  - So number of comparisons are 9.
- Time complexity is O(n)

- If the element is not found in array, then we have to search entire array, so it comes under worst case.

BINARY SEARCH



BEST CASE: Time complexity is O(1)
AVERAGE CASE: Time complexity is O(log n)
WORST CASE: Time complexity is O(log n)

Best Case Time Complexity

- For a given algorithm, T(n) is best case time complexity if algorithm have to repeat its basic operation for minimum time for given input size n.
- Determination of T(n) is called Best case time complexity analysis.

- When the key to be searched is present at first position
- Number of Comparisons: 1

$$\mathbf{T_{best}(n) = 1}$$

Example: 100, 500, 230, 45, 1, 7, 234

Key=100

Worst Case Time Complexity

- For a given algorithm, T(n) is worst case time complexity if algorithm have to repeat its basic operation for maximum number of times for given input size n.
- Determination of T(n) is called worst case time complexity analysis.
- When the key to be searched not present/present at last position
- Number of Comparisons: n

$$\mathbf{T_{worst}(n) = n}$$

Example: 100, 500, 230, 45, 1, 7, 234

Key=234 or key=5 (Any number not present in the array or present at end)

Average Case Time Complexity

- For a given algorithm, T(n) is average case time complexity if algorithm have to repeat its basic operation for average number of times for given input size n.
- Determination of T(n) is called average case time complexity analysis.
- When the key to be searched is present in the middle
- P ⇒ Probability of getting successful search
- n ⇒ Total number of elements in the list

$$T_{average}(n) = \frac{P(n+1)}{2} + n(1-P)$$

Example: 100, 500, 230, 45, 1, 7, 234

Key=230 or key=45

Asymptotic Notations

SCHOOL OF ENGINEERING

- The order of growth can be expressed using two methods:

1. Order of growth using asymptotic notations.
2. Order of growth using limits.

<u>Asymptotic Behavior of a Function</u>

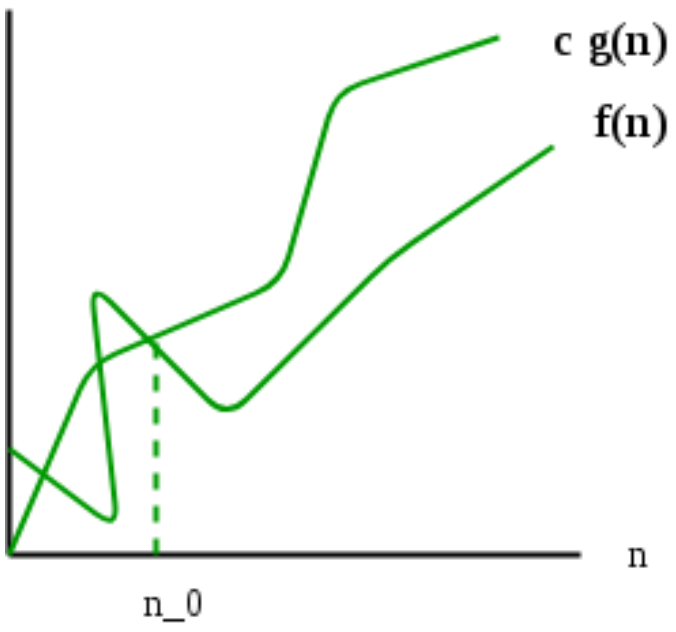
- Asymptotic notations are the notations using which two algorithms can be compared with respect to efficiency based on the order of growth of an algorithm's basic operation.

- The different types of asymptotic notations are:

1. O (Big Oh)
2. Ω (Big Omega)
3. Θ (Big Theta)

BIG-OH(O)

- Big O specifically describes the worst case scenario.
- It represents the upper bound running time complexity of an algorithm.
- Big-oh is the formal method of expressing the upper bound of an algorithm's running time. It is the measure of the longest amount of time.
- The function f (n) = O (g (n)) if and only if there exist positive constant c > 0and n_0 such that
- **$f(n) \leq c*g(n)\ \forall n \geq n_0$ and n_0>=1**



f(n) = O(g(n))

EXAMPLE

$O(g(n)) = \{f(n) : \exists$ positive constants c and n_0, such that $\forall n \geq n_0$, we have $f(n) \leq cg(n)\}$

Ex: f(n) = 2n+3

$2n+3 \leq 2n+3n \quad \forall n \geq n0$

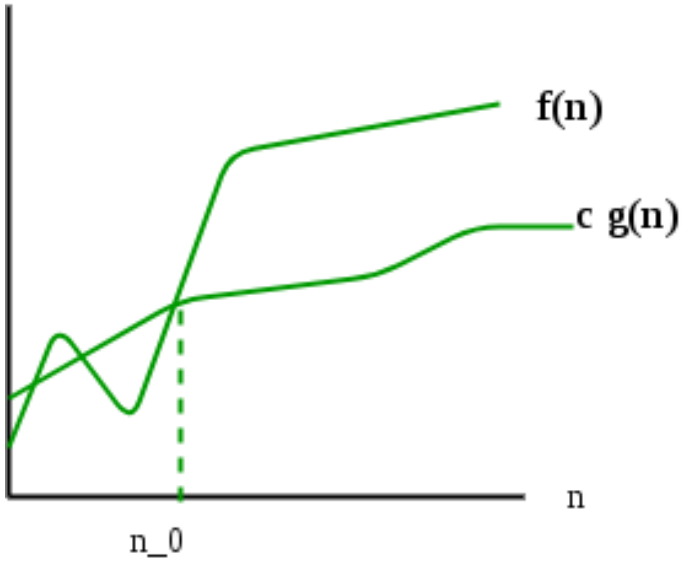
$\leq 5n \quad n \geq 1$

$\therefore f(n) = O(n)$

c = 5, n0=1, g(n) = n

BIG-OMEGA(Ω)

- Omega is the formal method of expressing the lower bound of an algorithm's running time.
- It is the measure of the smallest amount of time.
- The function f(n) = Ω(g (n)) if and only if there exist positive constant c>=1 and n_0 >1such that **f(n) ≥ c*g(n) ∀n ≥ n0**



f(n) = Omega(g(n))

EXAMPLE

$\Omega(g(n)) = \{f(n) : \exists$ positive constants c and n_0, such that $\forall n \geq n_0$, we have $f(n) \geq cg(n)$ $\}$

Ex: $f(n) = 2n+3$

$2n+3 \geq 1*n \qquad \forall n \geq n0$

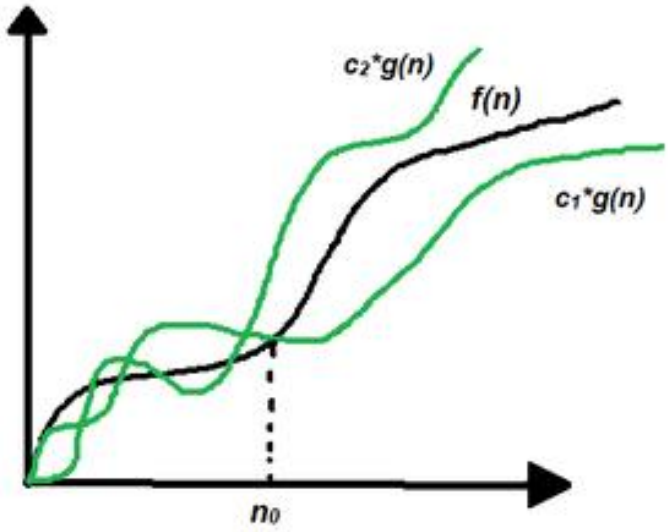
$\qquad \geq 1n \qquad \forall\ n \geq 1$

$\therefore f(n) = \Omega(n)$

$c = 1, n0=1, g(n) = n$

BIG THETA(Θ -notation)

- Theta is the formal method of expressing the average bound of an algorithm's running time.
- It is the measure of the average amount of time.
- The function f (n) = Θ(g (n)) if and only if there exist positive constants c1, c2 and n_0 such that $c1 * g(n) \leq f(n) \leq c2 * g(n)$ $\forall n \geq n_0$



EXAMPLE

$\Theta(g(n)) = \{f(n) : \exists$ positive constants c1, c2, and n0, such that $\forall n \geq n_0$, $c1g(n) \leq f(n) \leq c2g(n)\}$

Ex: $f(n) = 2n+3$

$1n \leq 2n+3 \leq 5n$

RELATIONS BETWEEN Θ, Ω, O

Theorem :

For any two functions g(n) and f(n), $f(n) = \Theta(g(n))$ iff $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

I.e., $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$

In practice, asymptotically tight bounds are obtained from asymptotic upper and lower bounds.

NO UNIQUENESS

- There is no unique set of values for n_0 and c in proving the asymptotic bounds
- Prove that 100n + 5 = O(n2)

$100n + 5 \leq 100n + n = 101n \leq 101n^2$

for all $n \geq 5$

$n^0 = 5$ and $c = 101$ is a solution

$100n + 5 \leq 100n + 5n = 105n \leq 105n^2$

for all $n \geq 1$

$n^0 = 1$ and $c = 105$ is also a solution

Must find SOME constants c and n_0 that satisfy the asymptotic notation relation

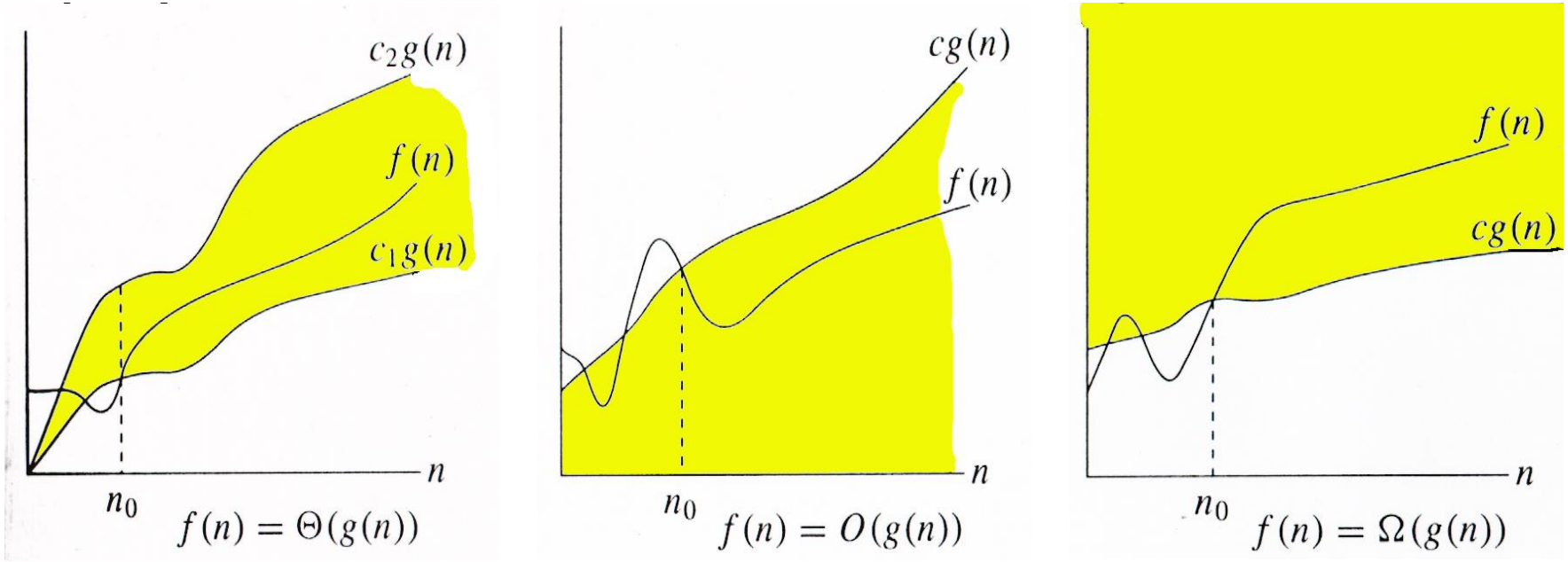
RELATIONS BETWEEN Θ, Ω, O

THEOREM : For any two functions g(n) and f(n),

f(n) = Θ(g(n)) iff

f(n) = O(g(n)) and f(n) = Ω(g(n)).

In practice, asymptotically tight bounds are obtained from asymptotic upper and lower



LITTLE OH NOTATION

Little-o notation is used to denote an upper-bound that is not asymptotically tight.

NOTE: In Big O, the functions f(n) and g(n) have the same rate of growth i.e. if f(n) is n^2 then g(n) is also n^2 .For f(n) to be O , f(n)<=g(n).

If f(n)=n^2, the relation holds true for g(n)= n3,n4,…. But in big O, tightest bound is chosen.

- o(g(n)) =

  {f(n) : for any positive constant c >0,

  there exists a constant $n_0 > 0$ such that

  0 <= f(n) < cg(n) for all n >= n_0}

Intuitively, $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$

LITTLE OH NOTATION

- little-o notation does not allow the function f(n) to have the same growth rate as g(n) .

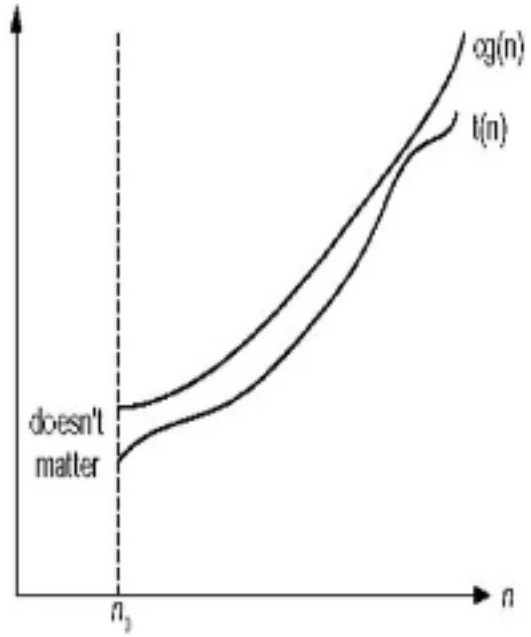
Example:

For $f(n) = 2n + 3$, we have:

- $\boldsymbol{f(n) = O(n)}$ but $\boldsymbol{f(n) \neq o(n)}$
- $f(n) = O(n^2)$ and $f(n) = o(n^2)$
- $f(n) = O(n^3)$ and $f(n) = o(n^3)$

In general, for $f(n) = a_x n^x + a_{x-1} n^{x-1} + \ldots + a_0$, we will have:

- $\boldsymbol{f(n) = O(n^x)}$ but $\boldsymbol{f(n) \neq o(n^x)}$
- $f(n) = O(n^{x+1})$ and $f(n) = o(n^{x+1})$
- $f(n) = O(n^{x+2})$ and $f(n) = o(n^{x+2})$



LITTLE OH NOTATION

1.If $f(n) = n^2$ and $g(n) = n^3$ then check whether $f(n) = o(g(n))$ or not.

Solution:

For $f(n)=o(g(n))$

Intuitively, $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

$$\lim_{n\to\infty} \frac{n^2}{n^3}$$

$$= \lim_{n\to\infty} \frac{1}{n}$$

$$= \frac{1}{\infty}$$

$$= 0$$

The result is 0, and it satisfies the equation mentioned above.

So we can say that $f(n) = o(g(n))$.

LITTLE OH NOTATION

2.Given f(n)=logn and g(n)=n prove that f(n)=o(g(n).

Hint: use Hospital's rule to prove it.

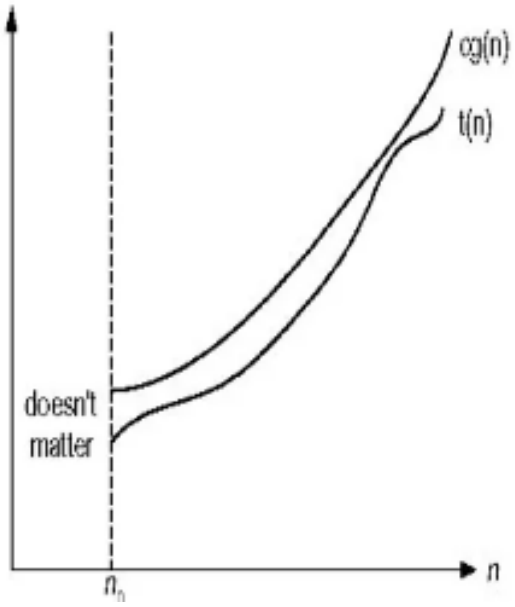
Refer class notes for solution.

LITTLE ω NOTATIONS

- denotes a lower bound that is not asymptotically tight
- $\omega(g(n)) = \{f(n)$: for any constant $c > 0$,

  $\exists$ a constant $n_0 > 0$

  such that $0 \le cg(n) < f(n), \forall n \ge n_0\}$
- Intuitively $\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$



LITTLE ω NOTATIONS

1.Given $f(n)=3^n$ and $g(n)=2^n$, prove that $f(n)= \omega(g(n))$

Solution:

$$\lim_{n\to\infty}\frac{f(n)}{g(n)}=\infty$$

Refer class notes for solution.

USING THE ORDER TO ESTIMATE RUN TIME

Efficiency: The efficiency of an algorithm is a measure of its run time. This is proportional to operation.

Size: The size is a measure of the complexity.(Usually the no of items in the list or input size)

Order: The order of an algorithm is a measure of the efficiency as a function of the size.

PROBLEMS ON EFFICIENCY

Problem 1:

A computer is programmed with an algorithm of Quadratic order, it takes 2 seconds to solve a problem of size 15. Estimate how long would it take to solve a problem of size 150.

Solution:

$E \propto n^2$ (E is proportional to n^2)

To remove proportionality use a constant K, so

$E=K.n^2$ (E=2, n=15)

$2=K(15)^2$

$2=K(225)$

$K=2/225$

If n=150

$E=(2/225)(150^2)$

E =200 sec

PROBLEMS ON EFFICIENCY

Problem 2:

A computer is programmed with an algorithm of 2^n order, it takes 2 seconds to solve a problem of size 15. Estimate how long would it take to solve a problem of size 150.

Solution:

$E \propto 2n$ (E is proportional to 2n)

To remove proportionality use a constant K, so

$E=K2^n$ (E=2, n=15)

$2=K(2^{15})$

$K=1/2^{14}$

If n=150

$E=(1/2^{14})(2^{150})$

$E=2^{136}$ seconds

PROBLEMS ON EFFICIENCY

Problem 3:

An algorithm has an order of n^4 and it takes 3750 seconds to solve a problem of size 5.

Estimate time taken to solve a problem of size 8.

Solution:

$E \propto n^4$ (E is proportional to n^4)

To remove proportionality use a constant K, so

$E=Kn^4$ (E=3750, n=5)

$3750=K(5^4)$

$K=6$

If $n=8$

$E=6(8^4)$

E =24576 seconds

PROBLEMS ON EFFICIENCY

Problem 4:

An algorithm has an order n and it takes 30 seconds to solve a problem of size 3. Estimate time taken to solve a problem of size 6.

Solution:

$E \propto n$ (E is proportional to n)

To remove proportionality use a constant K, so

$E=Kn$ ($E=30$, $n=3$)

$30=K(3)$

$K=10$

If $n=6$

$E=10(6)$

$E=60$ seconds

PROBLEMS ON EFFICIENCY

Problem 5: An algorithm has order 3n and it takes 324seconds to solve a problem of size 4. Estimate the time taken to solve a problem of size 11.

Solution:708588 Seconds

Example 6: An algorithm has order 5n and it takes 200seconds to solve a problem of size 2. Estimate the time taken to solve a problem of size 7.

Solution: 625000 Seconds

Example 7: An algorithm has order 7n and it takes 14seconds to solve a problem of size 1. Estimate the time taken to solve a problem of size 9.

Solution: 80707214 Seconds

SUMMARY OF ASYMPTOTIC NOTATIONS

Notation Name	Mathematical inequality	Meaning
θ	+ve constant c_1, c_2 and n_0 such that $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all n, $n \ge n_0$	Minimum and maximum time that a function f can take
O	+ve constant c and n_0 such that $0 \le f(n) \le cg(n)$ for all n, $n \ge n_0$	Maximum time that a function f can take
Ω	+ve constant c and n_0 such that $0 \le cg(n) \le f(n)$ for all n, $n \ge n_0$	Minimum time that a function f can take
o	$c > 0$ there exist $n_0 > 0$ such that $0 \le f(n) < cg(n)$ for all $n \ge n_0$	Function f will take strictly less than Maximum time
ω	$c > 0$ there exist $n_0 > 0$ such that $0 \le cg(n) < f(n)$ for all $n \ge n_0$	Function f will take strictly greater than Maximum time

Mathematical Analysis for Non-Recursive Algorithms

1. Decide on a parameter indicating an **input's size.**

2. Identify the **basic operation.**

3. Does C(n) depends **only on n** or does it also **depend on type** of input?

4. **Set up sum** expressing the # of times basic operation is executed.

5. Find closed form for the sum or at least establish its order of growth

Problems

1. Maximum of n elements
2. Element uniqueness problem
3. Matrix multiplication
4. Linear Search

1. Maximum of n elements

ALGORITHM MaxElement(A[0..n-1])

```
maxval <- A[0]
for i =1 to n-1 do
        if A[i] > maxval
                maxval =A[i]
return maxval
```

Input size: n
Basic operation: Comparison (>)

$$C(n) = \sum_{i=1}^{n-1} 1$$

$$= n - 1 \cancel{-1} \cancel{+1}$$

$$C(n) = n - 1 \simeq O(n)$$

2. Element uniqueness problem

SCHOOL OF ENGINEERING

Input size: n

Basic operation: Comparison

ALGORITHM UniqueElements(A[0..n-1])

```
for i = 0 to n-2 do
        for j =i+1 to n-1 do
                if A[i] = A[j]
                        return false
return true
```

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

$$= \sum_{i=0}^{n-2} n-1-i \cancel{-1} \cancel{+1}$$

$$= \sum_{i=0}^{n-2} n-1-i$$

$$i \Rightarrow 0, 1, 2 \ldots n-2$$

$$= (n-1) + (n-2) + \ldots$$

$$= \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2} \simeq n^2$$

$$C(n) = O(n^2)$$

3. Matrix Multiplication

ALGORITHM MatrixMultiplication(A[0..n-1, 0..n-1], B[0..n-1, 0..n-1])

```
for i = 0 to n-1 do
        for j =0 to n-1 do
                C[i, j] = 0.0
                for k = 0 to n-1 do
                        C[i, j] = C[i, j] + A[i, k]×B[k, j]
return C
```

$$C(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

$$= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n - \cancel{1} + \cancel{1}$$

$$= \sum_{i=0}^{n-1} n - \cancel{1} + \cancel{1} \cdot n$$

$$= n - \cancel{1} + \cancel{1} \cdot n \cdot n$$

$$= n^3$$

$$C(n) \in \Theta(n^3)$$

Input size: n

Basic operation: Multiplication (*)

4. Linear Search

ALGORITHM LinearSearch(key, A[0..n-1], n)

```
for i = 0 to n-1 do
        if(key==a[i])
                return i;
return -1
```

$$C(n) = \sum_{i=0}^{n-1} 1$$

$$= n - \cancel{1} - 0 + \cancel{1}$$

$$= n$$

$$C(n) \in \Theta(n)$$

Best case $\Rightarrow \Omega(1)$ [Ele present @ 1st place]

Worst case $\Rightarrow O(n)$ [Ele present @ last / not present]

Average case $\Rightarrow \Theta(n)$

Input size: n

Basic operation: Comparison (==)

Mathematical Analysis for Recursive Algorithms

1. Decide on a parameter indicating an **input's size**

1. Identify the **basic operation**

1. Does C(n) depends **only on n** or does it also **depend on type** of input?

1. **Set up Recurrence relation** expressing the number of of times basic operation is executed.

1. Solve the recurrence relation and establish its order of growth

Types of Recursion

1. **Direct Recursion:** A recursive function that invokes itself is said to have direct recursion.

1. **Indirect Recursion:** A function contains a call to another function which in turn calls another function which in turn calls another function and so on and eventually calls the first function is called indirect recursion.

Basic Terms

1. **Base Case:** The base case is a way to return without making a recursive call.

1. **General Case:** The recursive function in the program is known as general case.

Types

1. Substitution Method

1. Master Theorem

1. Recurrence Tree

Problems

1. Factorial of a number

1. Tower of Hanoi

1. Factorial of a Number

ALGORITHM Fact(n)

```
if(n==0)
        return 1;
else
        return n*Fact(n-1);
```

$$M(n) = \begin{cases} 0 & \text{if } n = 0 \\ M(n-1)+1 & \text{otherwise.} \end{cases}$$

$$M(n) = M(n-1) + 1$$

Applying Backward substitution method.,

$$M(n) = [M(n-2)+1]+1$$

$$= [M(n-3)+1]+2$$

$$= [M(n-3)]+3$$

$\therefore$ We can derive.,

$$M(n) = M(n-i)+i$$

Substituting $i = n$.,

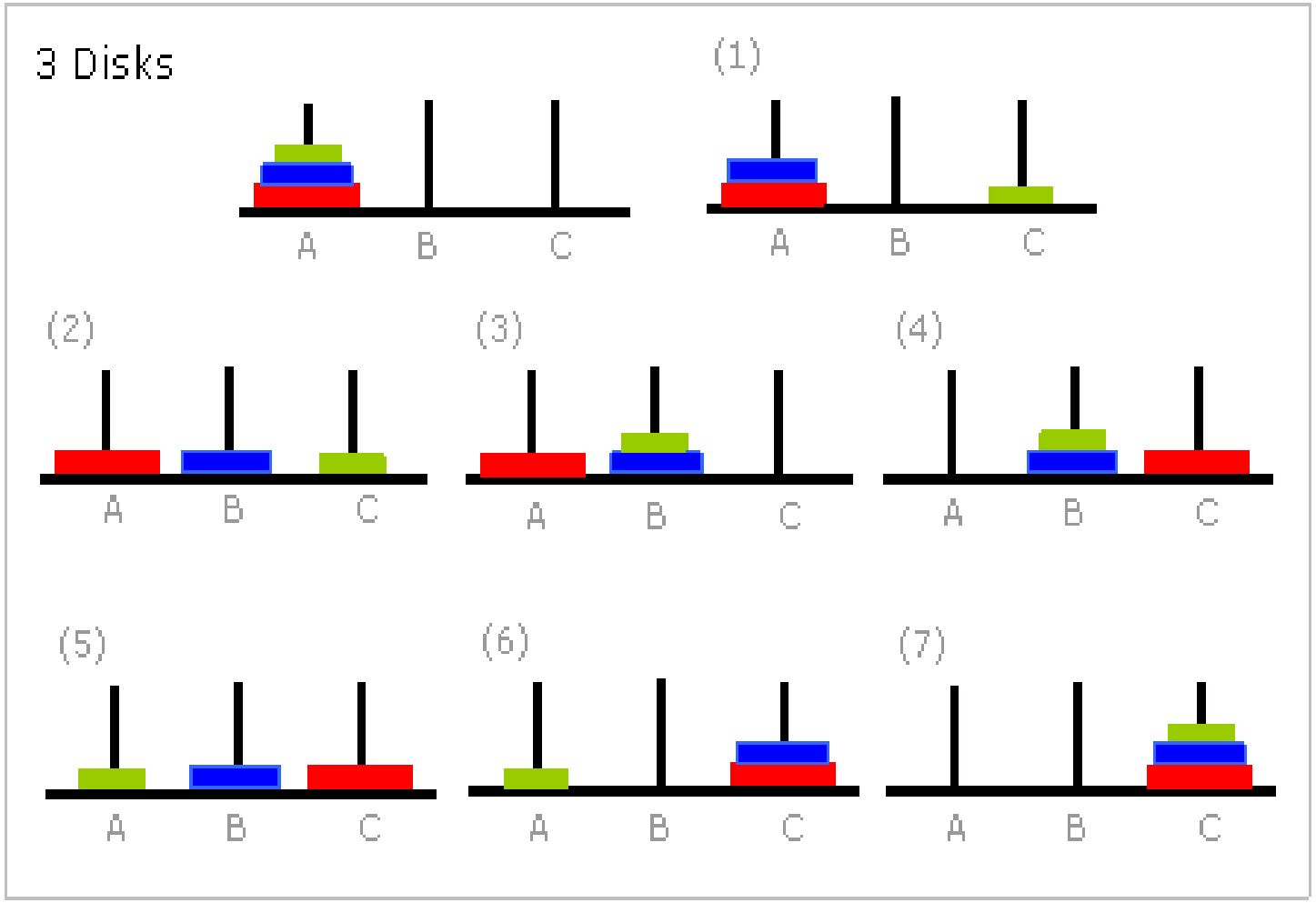
$$M(n) = M(\cancel{n}-\cancel{n})+n$$

$$= M(0)+n$$

$$M(n) = n$$

$$\boxed{\therefore f(n) \in \Theta(n)}$$

2. Tower of Hanoi



Step 1 – Move n-1 disks from source to aux
Step 2 – Move nth disk from source to dest
Step 3 – Move n-1 disks from aux to dest

2. Tower of Hanoi

```
Algorithm Hanoi(n, source, dest,
aux)
IF n == 1, THEN
    move disk from source to
dest ELSE
    Hanoi(n - 1, source, aux, dest)    // Step 1
    move disk from source to dest       // Step 2
    Hanoi(n - 1, aux, dest, source)    // Step 3
END IF
```

$$M(n) = \begin{cases} 1 & \text{if } n = 1 \\ M(n-1)+1+M(n-1) & \text{otherwise} \end{cases}$$

Applying Backward Substitution Method.,

$$M(n) = M(n-1) + 1 + M(n-1)$$
$$= 2M(n-1) + 1$$
$$\Rightarrow 2[2M(n-2)+1]+1$$
$$= 2^2 M(n-2) + 2 + 1$$
$$\Rightarrow 2^2[2M(n-3)+1] + 2 + 1$$
$$= 2^3 M(n-3) + 2^2 + 2 + 1$$

$\therefore$ We can derive.,

$$M(n) = 2^i M(n-i) + \underbrace{2^{i-1} + 2^{i-2} + \cdots + 2^0}$$
$$M(n) = 2^i M(n-i) + 2^i - 1$$

Substituting $i = n-1$

$$M(n) = 2^{n-1} M(n-(n-1)) + 2^{n-1} - 1$$
$$= 2^{n-1} M(\cancel{n} - \cancel{n} + 1) + 2^{n-1} - 1$$
$$= 2^{n-1} M(1) + 2^{n-1} - 1$$
$$= 2 \cdot 2^{n-1} - 1$$
$$M(n) = 2^n - 1 \qquad \boxed{\therefore f(n) \in \Theta(2^n)}$$

Thank You